

Baseline 带宽语义、2 NPU × 2 SSU 实验与 128 NPU 稳态验证

SSD40 → NPU50 离散事件模型审计与滚动实验记录

qos_storage_sim 实验审计

2026-08-30

目录

1 执行摘要	2
2 Baseline 的真实语义	3
2.1 它做了什么	3
2.2 为什么轮流跑 40，长期平均可以是 20	3
2.3 为什么带宽不是均匀的，NPU 计算仍可连续	4
3 2 NPU × 2 SSU：可手算案例	4
3.1 案例 A：均匀满载	5
3.2 案例 B：同盘热点，但 fleet 总容量看起来够	5
3.3 案例 C：互补放置	5
3.4 案例 D：总需求恰好 40，但大命令造成 barrier bubble	6
4 2 × 2 离散事件实测结果	6
4.1 容量与利用率	6
4.2 TTFT 与 barrier	6
5 当前 SSD40 → NPU50 模型审计	7
5.1 “每个 I/O 以 40 GB/s 执行”是事实，但不是多条各享 40	7
5.2 CIR 的意义	7
5.3 NPU50 是独立的第二级单服务器	8
5.4 哪些是正确抽象，哪些是待补能力	8
6 32 NPU 设计收敛与 128 NPU 验证路径	9
6.1 32 NPU 的已验证锚点	9
6.2 Adaptive V2.1 为什么需要自适应 residual	9
6.3 为什么没有继续放大 EDF/barrier-first V3	10
6.4 数据面敏感性实验	10
6.5 128 NPU 容量点为什么选 24/40/70 SSU	10
6.6 冻结比较矩阵与版本关系	11
6.7 完成条件与判断顺序	12
6.8 有限 saturated prefix 为什么可代表测量窗口内的无限 tail	12

7 128 NPU 最终配对结果	13
7.1 主结果	13
7.2 与同 SSU baseline 的配对差值	14
7.3 25 ms 与 50 ms 的性能/写入折中	15
7.4 128 与 32 NPU 不一致的已定位原因	15
8 对“远优于 baseline”的物理解释	15
9 复现与文件索引	16

1 执行摘要

这份报告回答三个容易混在一起的问题：baseline 为什么在 NPU 长期满载时仍能得到很高的平均 NPU 利用率；当前仿真中“被调度的 I/O 以 40 GB/s 执行”到底是什么意思；以及怎样把 32 NPU 的新 Scheme B 结论扩展到真实 data、128 NPU 和不同 SSU 数量。

结论先行：

1. **Baseline 并没有按 NPU 均匀分配带宽。**它把一块 SSU 上的所有 I/O 都放到 Path0；Path0 内按提交顺序排队。对称、细粒度、长期饱和的输入会让不同 NPU 获得接近相同的长期吞吐，但这是一种输入与时间线的结果，不是 baseline 提供的公平性保证。
2. **高平均 NPU 利用率不等于每个时刻每个 NPU 都拿到相同带宽。**只要下一层数据能在当前层计算结束前到达，NPU 就可以连续计算。SSD 命令可以在多个 NPU 之间轮流以 40 GB/s 执行，而每个 NPU 的长期平均份额只有 20 GB/s；这两件事并不矛盾。
3. **当前代码没有把多条并发 I/O 各自都算成 40 GB/s。**每块 SSD 同时最多只有一条 active command；该命令以 40 GB/s 完成，因此单盘总服务率不会超过 40 GB/s。CIR/WRR 决定的是下一条命令选谁，而不是把正在执行的命令降到某个 CIR。
4. 这套模型在容量记账上自治，但有重要的真实性边界：单盘单命令、命令不可抢占、无固定 I/O 延迟、无队列深度吞吐曲线、整条命令完成后才进入 NPU50 队列，以及**有限 PIR 尚未实现**。2×2 合成 stress 已证明“大命令”拆分粒度会改变 barrier；不过真实 data 的命令只有 2.10–4.20 μs SSD 服务时间，所以不可抢占粒度本身不能解释真实实验中几十毫秒的差异。
5. **128 NPU Adaptive V2.1 在 SSU24/40/70 三点都形成强 SLO 改善，并保持 mean utilization。**相对配对 baseline 分别是 -0.25/+27.10、-0.03/+32.45、+1.92/+21.54 pp (util/SLO)。它在 SSU24 自动使用 V2 explicit spill，在 SSU40/70 自动保留 V1 coflow residual。
6. **50 ms 是明确的控制开销折中。**相比 25 ms，它在 SSU40/70 将 evaluation 大致减半、Path writes 降低约 49%/41%，但 SLO 分别下降 7.22/6.62 pp；SLO-first 默认仍应选 25 ms。
7. 32→128 不是简单比例缩放：ring ownership 的固定偏斜在更多 SSU 时更大，部分窗口也存在 outstanding queue drift，必须把 placement 与稳态诊断一并报告。

本报告的 2 NPU × 2 SSU、32 NPU 和 128 NPU 正式表均已完成。所有纳入主结论的行通过 summary invariant，所用 runner 的已选 case 完成且 source/config fingerprint 稳定；跨策略比较的 assignment/workload/placement/trace/simulator-input 指纹一致。SSU16 的独立 invalid 诊断不混入正式均值。

2 Baseline 的真实语义

2.1 它做了什么

Baseline 的客户端规则非常简单：对每块 SSU，所有 I/O 都固定选择 Path0，并且不读取 Path pressure。当前 FINAL_STATIC 把 SS 类的 20 GB/s CIR 预算分到 8 个 group、每组 12 条 SS Path，因此 Path0 的名义 CIR 只有：

$$\text{CIR}_{\text{Path0}} = \frac{20}{8 \times 12} = 0.20833 \text{ GB/s.}$$

但 CIR 不是“它最多只能得到的带宽”；PIR 为无穷，SSD 调度又是 work-conserving。当 Path0 是该 SSD 的唯一活跃 Path 时，全部 surplus 都回到它：

$$0.20833 + (40 - 0.20833) = 40 \text{ GB/s.}$$

这个 40 GB/s 属于**整个 Path0 队列当前被选中的一条命令**，不是每个 NPU 各自获得 40 GB/s，也不是控制器显式做了 per-NPU 均分。

可以把一块 SSD 上的 baseline 理解为：

所有 NPU 的 I/O → 同一个 Path0 FIFO → 单个 SSD40 后端
每次只执行一条命令

因此 baseline 没有下面这些特性：

- 没有“每个 NPU 固定分到 40 / 活跃 NPU 数”的寄存器配置；
- 没有 request、layer 或 TTFT deadline 感知；
- 不会把另一块空闲 SSU 的带宽搬给热点 SSU；
- 不保证短命令不会被前面的大命令阻塞；
- 不保证任何有限时间窗口内的逐 NPU 公平。

Baseline 有的只是：一块 SSD 有工作时尽量不空闲，Path0 的命令按队列顺序接受服务。在完全对称的长期 trace 中，这通常会“看起来像均分”。

2.2 为什么轮流跑 40，长期平均可以是 20

先只看一块 SSD。假设 NPU0 和 NPU1 各有一条 0.1 GB 命令，Path0 FIFO 轮流执行：

时间	0 ms	2.5 ms	5.0 ms
SSD0	NPU0: 40 GB/s NPU1: 40 GB/s		
传输数据	0.1 GB	0.1 GB	

单条命令的服务时间为：

$$0.1 \text{ GB} / 40 \text{ GB/s} = 2.5 \text{ ms.}$$

在整个 5 ms 窗口中：

- SSD0 始终以 40 GB/s 工作，总共传了 0.2 GB；

- NPU0 只在前 2.5 ms 被服务，但其长期平均是 $0.1/0.005 = 20$ GB/s；
- NPU1 同理也是 20 GB/s；
- 任意时刻都只有一条命令使用 SSD0，不存在 $40 + 40 = 80$ GB/s 的单盘超发。

因此要分清：

量	本例数值	含义
active command 瞬时速率	40 GB/s	当前被 SSD 后端执行的那一条命令
每 NPU 长期平均 SSD 份额	20 GB/s	被选中时间比例乘以 40 GB/s
SSD 总长期吞吐	40 GB/s	单盘物理容量，不会因 NPU 数增加而增加

两块 SSD 可以各自同时执行一条命令，所以 2 SSU 的 fleet 上限是 80 GB/s；但每块 SSD 仍分别受 40 GB/s 限制。

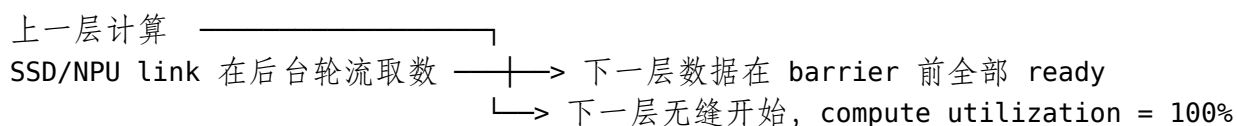
同样地，若两个独立 Path A/B 的目标长期份额是 30/10 GB/s，并且命令大小都为 1 GB，命令级调度可以形成重复的 A, A, B, A 时间线：

时间	0	25	50	75	100 ms
SSD	A@40	A@40	B@40	A@40	
数据	1 GB	1 GB	1 GB	1 GB	

100 ms 内 A 得到 3 GB，即长期 30 GB/s；B 得到 1 GB，即长期 10 GB/s。四条 active command 的瞬时速率全是 40 GB/s，但服务时间占比为 3:1，所以长期份额恰好是 30/10。CIR 在当前模型中控制的是这种**被选中机会**，不是把 A 的 active command 直接限成 30、把 B 的 active command 限成 10。

2.3 为什么带宽不是均匀的，NPU 计算仍可连续

NPU utilization 在实验中指 **compute busy time / measurement time**，不是 NPU link 的平均带宽比例。对某一层，只要它所需的所有 SSU 数据在上层计算结束前 ready，下一层就可以立刻开始计算。于是：



Baseline 可能让每个 NPU 的 I/O 在微观时间上交替突发，但只要这些突发都落在 compute slack 里，NPU 就看不到空洞。动态策略的主要机会不是凭空创造带宽，而是减少“数据总量已经传得差不多、但 barrier-critical 的最后一小块还在排队”的无效等待。

3 2 NPU × 2 SSU: 可手算案例

统一设置：2 NPU、2 SSU、每块 SSD40、每 NPU 独立 NPU50 link、8 层、每层计算 10 ms、batch=1、warm 满载。每层理想纯计算 TTFT 为 $8 \times 10 = 80$ ms；主 SLO 为 $2 \times$ ，即 160 ms。

令 $D_{n,s}$ 表示 NPU n 在连续计算时对 SSU s 的原始需求 (GB/s)。每层在该 SSU 上的数据量为 $D_{n,s} \times 0.01$ GB。

3.1 案例 A: 均匀满载

$$D = \begin{bmatrix} 20 & 20 \\ 20 & 20 \end{bmatrix} \text{ GB/s.}$$

每块 SSU 的总需求都是 $20 + 20 = 40$ GB/s, 恰好满载; 每个 NPU 的总接收需求也是 40 GB/s, 低于 NPU50。

实际实验把每个正 flow 拆成 4 条命令。于是每个 NPU、每块 SSU、每层的数据是 0.2 GB, 每条命令 0.05 GB; 一条命令在 SSD40 上耗时 1.25 ms。一块 SSU 每层共执行 $2 \times 4 = 8$ 条命令, 正好耗时 10 ms。两个 NPU 各自长期取得 20 GB/s/SSU, 下一层数据能在 10 ms compute 窗口内准备好。

结果: 两块 SSD 都是 100% busy, 两个 NPU link 都是 80% busy, 两个 NPU compute 都是 100% busy, TTFT 都是 80 ms。这里的“均匀”来自对称 workload、细粒度命令和长期重复, 不是 baseline 的 QoS 保证。

3.2 案例 B: 同盘热点, 但 fleet 总容量看起来够

$$D = \begin{bmatrix} 30 & 0 \\ 30 & 0 \end{bmatrix} \text{ GB/s.}$$

Fleet 总需求只有 60 GB/s, 小于 2 SSU 合计 80 GB/s; 但 SSU0 的需求是 60 GB/s, 超过单盘 40, SSU1 完全空闲。SSD 上的数据 placement 已经固定, 因此 SSU1 的 40 GB/s 不能代替 SSU0 读取数据。

对称长期服务下, 每个 NPU 从 SSU0 平均只能拿到约 20 GB/s, 相对所需 30 GB/s 的进度比例为:

$$20/30 = 66.67\%.$$

实测两个 NPU compute utilization 都是 66.67%, TTFT 都是 120 ms。注意 120 ms 仍小于 160 ms, 所以 $2 \times$ SLO 达标率仍是 100%。这个例子同时说明:

- fleet 总带宽足够不代表 placement 可行;
- SLO 达标率高不代表 NPU 已经满利用;
- 动态 CIR 不能让 SSU1 读取只存在于 SSU0 的数据。

3.3 案例 C: 互补放置

$$D = \begin{bmatrix} 38 & 0 \\ 0 & 38 \end{bmatrix} \text{ GB/s.}$$

NPU0 只读 SSU0, NPU1 只读 SSU1。每盘需求 $38 < 40$, 每个 NPU 的 link 需求 $38 < 50$ 。每层 SSD 时间为 $0.38/40 = 9.5$ ms, 能被 10 ms compute 隐藏。

实测两块 SSD 各 95% busy, 两个 NPU link 各 76% busy, 两个 NPU compute 都是 100%, TTFT 都是 80 ms。这是 placement 本身消除了冲突, 而不是 baseline 解决了冲突。

3.4 案例 D：总需求恰好 40，但大命令造成 barrier bubble

$$D = \begin{bmatrix} 30 & 0 \\ 10 & 0 \end{bmatrix} \text{ GB/s.}$$

SSU0 的长期原始需求恰好是 40 GB/s，fleet 和单盘容量从算术上都可行。若每个正 flow 每层只有一条命令，则 NPU0 的命令为 0.3 GB (7.5 ms)，NPU1 的命令为 0.1 GB (2.5 ms)。命令不可抢占，再叠加逐层 barrier 与 NPU link，某些时刻 SSD 队列无法保持理想交错，形成 head-of-line wait 和 SSD 空洞。

实测 NPU0/NPU1 compute utilization 为 72.29%/96.39%，平均仍有 84.34%；TTFT 为 110.67/83.00 ms。平均值掩盖了 NPU0 的明显退化。

保持需求矩阵、placement 和总字节完全不变，只把每个正 flow 拆成 4 条小命令后，两个 NPU compute 都恢复为 100%，TTFT 都恢复到 80 ms。这个对照证明：在命令达到 0.1–0.3 GB、不可抢占时间达到 2.5–7.5 ms 的合成 stress 中，**策略结果不仅取决于需求带宽，还取决于命令粒度和不可抢占时间**。它是模型敏感性测试，不应直接外推为真实 trace 的根因。

4 2 × 2 离散事件实测结果

测量窗口为 19,920 ms；所有 NPU 在整个窗口保持 saturated prefix，窗口内 admission 的请求全部 drain；SSD、NPU link、compute overlap 和服务量归属 invariant 全部通过。数值来自 results/baseline_2npu_teaching/results。

4.1 容量与利用率

Case	$D[\text{NPU0}; \text{NPU1}]$ 每个正 flow 的 GB/s	命令数	SSD util 0/1	NPU link util 0/1	Compute util 0/1
均匀满载	[20, 20]; [20, 20]	4	100.00% / 100.00%	80.00% / 80.00%	100.00% / 100.00%
同盘热点	[30, 0]; [30, 0]	4	100.00% / 0.00%	40.00% / 40.00%	66.67% / 66.67%
互补放置	[38, 0]; [0, 38]	16	95.00% / 95.00%	76.00% / 76.00%	100.00% / 100.00%
异构大命令	[30, 0]; [10, 0]	1	78.31% / 0.00%	43.37% / 19.28%	72.29% / 96.39%
异构小命令	[30, 0]; [10, 0]	4	100.00% / 0.00%	60.00% / 20.00%	100.00% / 100.00%

4.2 TTFT 与 barrier

Case	Mean compute util	TTFT NPU0/NPU1	Barrier wait NPU0/NPU1	2 × SLO NPU0/NPU1
均匀满载	100.00%	80.00 / 80.00 ms	0.00 / 0.00 ms	100% / 100%

Case	Mean compute util	TTFT NPU0/NPU1	Barrier wait NPU0/NPU1	2× SLO NPU0/NPU1
同盘热点	66.67%	120.00 / 120.00 ms	40.00 / 40.00 ms	100% / 100%
互补放置	100.00%	80.00 / 80.00 ms	0.00 / 0.00 ms	100% / 100%
异构大命令	84.34%	110.67 / 83.00 ms	30.67 / 3.00 ms	100% / 100%
异构小命令对 照	100.00%	80.00 / 80.00 ms	0.00 / 0.00 ms	100% / 100%

这些结果给出 baseline “看起来很好” 的三个条件：容量约束基本可行、输入/placement 足够对称、命令足够细而且 phase 能形成稳定流水。破坏其中任一条件，baseline 就可能出现热点、head-of-line 或逐 NPU 不均衡。

5 当前 SSD40 → NPU50 模型审计

5.1 “每个 I/O 以 40 GB/s 执行” 是事实，但不是多条各享 40

当前 SSD 调度器的实际顺序是：

1. 找出有 backlog 的 Path；
2. 由 CIR、组权重和 Path 权重计算**虚拟长期服务率**；
3. 用 `command_size / virtual_rate` 更新虚拟完成标签，选择下一条 Path；
4. 从该 Path 取出一条命令；
5. 该命令不可抢占地以 `disk_bw = 40 GB/s` 执行，服务时间是 `size / 40`；
6. 命令完成后再选择下一条。

代码同时检查每块 SSD 的 active flow 数不超过 1。因此 current model 是 command-level time division，不是多个 I/O 的 fluid sharing。它满足下面的容量恒等式：

$$\sum_n \text{served}_{n,s}(T) \leq 40T, \quad \forall s.$$

所以 “active I/O 的 `flow_bw=40`” 本身不是重复计算带宽的 bug。真正需要验证的是：真实 SSU 是否适合用 “单命令全速、命令之间时间复用” 来近似。

5.2 CIR 的意义

当前 CIR 更像命令仲裁权重，而不是 active 命令的物理限速器。例：两条始终 backlogged 的 Path，希望长期得到 30/10 GB/s；调度器可以让二者被选中的服务时间大致为 3:1，而被选中的每条命令仍以 40 GB/s 执行。

动态更新 CIR 时，已经在执行的命令不会被抢占，也不会重新计算完成时间。新 CIR 只影响后续命令的选择。因此控制器即使每 25 ms 更新，最坏生效延迟仍包含当前命令的剩余服务时间；大命令会降低控制灵敏度。

5.3 NPU50 是独立的第二级单服务器

SSD 命令完成后，数据才进入对应 NPU 的 FCFS link 队列。每个 NPU 同时最多接收一条 flow；active flow 以 50 GB/s 执行，服务时间为 $\text{size} / 50$ 。不同 NPU 有独立 link，所以 fleet NPU link 上限是 $\text{NPU 数} \times 50$ ，但单个 NPU 不能超过 50 GB/s。

这同时意味着模型是整条命令的 store-and-forward：同一 flow 不能一边从 SSD 流出、一边进入 NPU。真实 DMA 若能流式流水，当前模型可能高估 barrier；反过来，真实链路若还有共享 PCIe/switch bottleneck，当前“每 NPU 独立 50”又可能过于乐观。

正式 summary 保存的 measurement_npu_link_utilizations 是 **NPU50 的利用率比例，不是 GB/s**。换算关系是 $\text{utilization} \times 50 \text{ GB/s}$ 。Adaptive V2.1 的观测最大值分别为：SSU24 21.12% = 10.56 GB/s、SSU40 24.83% = 12.41 GB/s、SSU70 38.82% = 19.41 GB/s；对应 baseline 为 14.61% = 7.31 GB/s、22.23% = 11.11 GB/s、38.69% = 19.34 GB/s。因此 **NPU50 不是这些已完成行的可见瓶颈**。某些 SS 类请求的 raw demand 虽然高于 50，但端到端实际 service 已先受 SSD placement/排队与 admission 限制，不能仅凭 raw demand 宣称 NPU link 饱和。

5.4 哪些是正确抽象，哪些是待补能力

项目	当前实现	判断
单盘总吞吐上限	一条 active command $\times 40 \text{ GB/s}$	容量记账自治，不会单盘超发
CIR/WRR	形成虚拟完成标签，决定下一条命令	合理的命令级近似，但不是 fluid rate limiter
动态 CIR	原子更新，下一次仲裁生效	自治；大命令时控制生效会滞后
PIR	QoS 构造时只允许无穷 PIR	实际功能缺口：无法实验有限 PIR
SSD 并发	每盘最多一条 active I/O	真实性风险：未建模多 channel/queue 并行
SSD 服务时间	仅 $\text{size} / 40$	真实性风险：无固定 latency、随机性、QD/size 曲线
命令抢占	不支持	会产生 head-of-line；是否符合设备需实测校准
NPU link	每 NPU 单队列、active flow 50 GB/s	单 NPU 容量自治；没有共享 fabric bottleneck
SSD→NPU	整条 SSD 命令完成后再进 link	真实性风险：未建模 streaming/pipeline

5.4.1 真实 data 的命令很小

四个正式 workload profile 的 placement 使用 128-token block。实际 block 大小范围为：

$$0.0000839233\text{--}0.0001678467 \text{ GB,}$$

在 SSD40 上对应的单命令不可抢占时间只有：

$$2.098\text{--}4.196 \mu\text{s}.$$

因此 2×2 “异构大命令”中 7.5 ms 的 head-of-line 现象是故意构造的 stress，不是正式 trace 的命令尺度。真实实验中几十毫秒的 TTFT/SLO 差距更可能来自大量小命令的累计排队、固定 placement 热点、layer barrier、控制目标与控制陈旧，而不是某一条 $4.2 \mu\text{s}$ 命令不可抢占；已完成 128 行又排除了 NPU50 作为当前可见主瓶颈。单命令模型仍值得做 fluid sensitivity，但其优先级低于检查上述累计效应。

5.4.2 一个确定的输入校验 bug: NaN 会绕过比较

StaticQoSConfig 和 runtime update_path_cirs 当前用 $<0, >\text{disk_bw}, \text{CIR}>\text{PIR}$ 做范围检查，但没有先要求所有 CIR/weight 都是 finite。IEEE NaN 与这些数值比较都返回 false，因此 NaN CIR、PIR 或 weight 可能绕过构造/更新校验，随后污染虚拟完成标签。

这是一个应修复并补单测的健壮性 bug。正常控制器生成的是有限 grant，已完成实验的容量/服务归属 invariant 也全部通过，目前没有证据表明 NaN 进入了已发布结果；但正式 128 runner 应在 worker 启动前和每次 CIR commit 时显式断言 isfinite，不能只依赖末端统计发现问题。

因此对“当前 QoS 仿真程序有没有问题”的准确回答是：**没有发现把并发 I/O 各算 40 GB/s 的容量 bug；但 finite PIR 是实际缺口，NaN 校验是实际 bug，单命令与 store-and-forward 是需要硬件/替代模型验证的真实性假设。** 2×2 大/小命令对照显示 15.66 个百分点的 mean compute utilization 差异，但真实命令是微秒级，不能用这个合成结果解释真实 trace 的几十毫秒差距。

6 32 NPU 设计收敛与 128 NPU 验证路径

6.1 32 NPU 的已验证锚点

32 NPU、真实四类 profile、16 层 warm steady-state 的正式结果给出了扩展到 128 NPU 前的 paired anchor:

SSU	Baseline mean compute util	Baseline equal-NPU $2 \times$ SLO
6	32.69%	25.00%
10	57.24%	75.00%
18	93.29%	79.38%

Adaptive Admission V2.1 的 25 ms 正式结果覆盖三个容量点:

SSU	Baseline util / SLO	Adaptive V2.1 util / SLO	配对变化 (按显示值)
6	32.69% / 25.00%	33.01% / 75.24%	util +0.32 pp, SLO +50.24 pp
10	57.24% / 75.00%	57.00% / 84.57%	util -0.24 pp, SLO +9.57 pp
18	93.29% / 79.38%	93.38% / 98.85%	util +0.09 pp, SLO +19.47 pp

这说明新策略在 32 NPU 中的主要收益是跨过 binary SLO threshold, 同时基本保持容量受限的 mean utilization; 它不是通过创造额外吞吐取得收益。

6.2 Adaptive V2.1 为什么需要自适应 residual

V1 和 V2 使用相同的 cost-first SLO admission set, 区别只在 selected floor 与 rejected background 之后如何消费剩余带宽:

- V1 使用 request-proportional coflow residual, 在中度/轻度过载时保持完整 request 的各 SSU 进度协调;
- V2 使用 selected-first explicit spill, 在严重过载时优先填满可帮助已选请求的 SSD 列, 避免 capacity tail 分给当期无法跨过 SLO threshold 的请求;

- Adaptive V2.1 仅用当前 active manifest 的 selected / active 比例做因果选择：比例严格低于 0.75 用 V2 explicit spill，否则保留 V1 coflow residual；入选 request 仍 pin 到完成。

0.75 是工作点启发式，不是通用最优值。32×6 的严重过载点主要使用 explicit 模式（243/245 次 evaluation），32×10 和 32×18 则全部使用 coflow residual（154/154、126/126），从而得到上表三个点。128 NPU 的最终 mode counts 与设计一致：SSU24 为 explicit/coflow=248/1，SSU40 为 0/166，SSU70 为 0/131；三个 case 均完成，source/config fingerprint 稳定。

6.3 为什么没有继续放大 EDF/barrier-first V3

V3 尝试每 25 ms 按 deadline、barrier wait 和 laxity 优先，但 32 NPU × SSU10 的完整配对结果是：

策略	Mean NPU util	Equal-NPU SLO	相对 baseline
baseline	57.24%	75.00%	—
V1 admission25	57.00%	84.57%	-0.24 pp util / +9.57 pp SLO
EDF/barrier-first V3	56.68%	74.92%	-0.56 pp util / -0.08 pp SLO

静态快照中 V3 先选择 deadline 更紧但 footprint 很大的 SS 请求，只能保护 8 个请求；cardinality-first admission 可保护 26 个。V3 的 SSD mean utilization 仍约 95.32%，所以它的负结果不是 SSU 没用满，而是 EDF priority 不等价于最大化 binary-SLO pass count。该版本不值得运行 128 NPU，设计已收敛为 Adaptive V2.1 的 cardinality-first/pinned admission。

6.4 数据面敏感性实验

建议保留当前模型作为 Model A，并至少增加一个 Model B：

- **Model A — 当前命令模型：**每盘单 active command、40 GB/s、不可抢占；
- **Model B — 离散时间 fluid/processor-sharing：**每 1 ms 或更粗的安全量子重新计算活跃 Path 的 CIR/PIR/WRR 速率，多 flow 可同时推进，但总和不超过 40；
- **Model C — 校准模型（有硬件数据时）：**固定命令 latency + size/QD 吞吐曲线 + 有限并行度 + 可选 streaming SSD→NPU。

最低限度应做四个 sensitivity sweep：

1. 同一 trace 将每个 flow 拆成 1/2/4/8/16 条命令；
2. current command model 与 fluid model 的策略排名比较；
3. NPU50 store-and-forward 与 streaming 两种模式比较；
4. 有限 PIR、无限 PIR 两种控制边界比较。

若新 Scheme B 在这些模型下相对 baseline 的方向一致，结论才具有较强鲁棒性；若策略排名翻转，问题主要是策略依赖 command granularity，而不是单纯的 SSU 总带宽不足。

6.5 128 NPU 容量点为什么选 24/40/70 SSU

此前 32 NPU 真实 trace 的 fleet raw demand 约为 666.236 GB/s。按 4 倍 NPU 线性放大，128 NPU 的 fleet raw demand 约为 2664.944 GB/s。每块 SSU40，因此 fleet raw-capacity knee 约为：

$$2664.944/40 = 66.624 \text{ SSU.}$$

128-NPU 配置	总 SSD 容量	对应 32-NPU 容量比例	主要目的
SSU24	960 GB/s	SSU6	严重容量受限；验证 admission 的 SLO cardinality 与公平性代价
SSU40	1600 GB/s	SSU10	中度容量受限；验证瞬时 hotspot 与目标函数
SSU70	2800 GB/s	SSU17.5	略过 fleet raw knee；验证 placement、NPU50、barrier 与调度开

总容量跨过 knee 不等于每块盘都可行。SSU70 仍必须检查每盘 workload、最大/分位 SSD utilization、热点持续时间与固定 placement。

6.5.1 32→128 容量比例相同，但 ring placement 不等价

当前 consistent-hash ring 每个 SSU 有 256 个 virtual node。对完整 hash 区间做精确积分后，最大 ownership 偏斜随 SSU 数增加而变大：

对应容量点	32 NPU ring max ownership	128 NPU ring max ownership	结果
$32 \times 10 \leftrightarrow 128 \times 40$	$1.0555 \times$	$1.1696 \times$	128 热点盘比理想均值高
$32 \times 18 \leftrightarrow 128 \times 70$ (容量近似)	$1.0592 \times$	$1.1504 \times$	128 热点盘比理想均值高

128×40 的 full-prefix manifest SSD CV 为 0.0670， 32×10 只有 0.0319； 128×40 的持久队列热点 SSU4 正好也是 ring ownership 最大的 SSU4。于是相同 NPU/SSU 比例并不保持每请求 fanout、ownership 分布或热点严重程度，这解释了 baseline 从 32×10 的 57.24% util 降到 128×40 的 51.60%，也说明 32 NPU 数值只能作为容量锚点，不能作为 128 NPU 的直接预测。

6.5.1.1 下一步 placement-only 反事实：增加 virtual nodes 精确 ring 区间积分还评估了 $256 \rightarrow 1024 \rightarrow 4096$ virtual nodes/SSU，只改变 placement ring 密度：

SSU	256 vnode max / stddev	1024 vnode max / stddev	4096 vnode max / stddev
40	$1.169584 \times / 0.066797$	$1.058502 \times / 0.026771$	$1.034628 \times / 0.015789$
70	$1.150396 \times / 0.058924$	$1.077290 \times / 0.030796$	$1.033362 \times / 0.012687$

这说明增加 vnode 很可能降低固定 ownership hotspot，值得作为下一轮 placement 实验。**它只是尚未运行端到端 DES 的反事实，没有混入本轮任何 workload/placement fingerprint，也不能用来重解释本报告的 256-vnode 数值。**真正验证需要用新 placement hash 成对重跑 baseline 与 Adaptive。

6.6 冻结比较矩阵与版本关系

使用真实 data、128 NPU、16 层、batch=1、warm steady-state、同一 seed、同一 request prefix、同一 placement 和同一测量窗口，比较：

策略	控制语义	作用
baseline	固定 Path0, 无 pressure read	成对基线
current Scheme B	causal max-min	证明旧目标函数的差异
V1 admission 25 ms	cost-first admission + coflow residual	中/轻度过载主策略
V2 admission 25 ms	同一 admission set + explicit selected-first spill	严重过载 residual 对照
Adaptive V2.1 25 ms	selected fraction <0.75 用 V2, 否则用 V1	跨容量统一策略; 128 三点正式完成
admission 50 ms	batch-membership 事件触发, 最小间隔 50 ms	较低写入频率/利用率配置

25/50 ms 是 event-gated 最小间隔, 不是固定周期 timer。每个 case 必须记录源码、配置、workload、placement 和输入 fingerprint, 并拒绝不同 fingerprint 的缓存结果。

6.7 完成条件与判断顺序

正式结果只有在以下 invariant 全部成立时才可发表:

- 所有 128 个 NPU 都完成 warmup, 测量窗口内每个 NPU 都有 SLO 样本;
- window admission 全部 tagged, tagged request 全部 drain;
- drain 期间任何 NPU 的 saturated prefix 都没有提前耗尽;
- 每盘同时最多一条 active command, 服务量与 busy time 一致;
- 每 NPU link 的服务量、busy time 和 50 GB/s 上限一致;
- measurement duration、compute overlap、SSD overlap、link overlap 全部通过;
- outstanding queue 在窗口头尾的变化必须单列, 不能把明显非稳态误称为 steady-state。

判断顺序应是:

1. 先判有效性与稳态;
2. 再比较 equal-NPU SLO、mean/min/per-NPU compute utilization;
3. 再看 request-weighted TTFT、P99、throughput 和各类别;
4. 最后结合每盘 demand/served、queue wait 和控制 writes 解释差异。

不能只看一个 fleet mean: 它会同时掩盖热点盘、慢 NPU 和少数类别退化。

6.8 有限 saturated prefix 为什么可代表测量窗口内的无限 tail

正式 workload 为每个 NPU 预生成 32 个请求, batch=1; 它不是用 “所有请求最终跑完” 近似无限时间, 而是给有限 warmup、measurement 和 tagged-drain 窗口提供一个足够长的 guard suffix。no_backlog_exhaustion=true 保证从 warmup 到最后一个 tagged request drain 完成, 没有任何 NPU 触碰这 32 个请求的尾边界。

离散事件仿真是因果的: 某时刻以前的事件只取决于此前已经 admission/提交的请求和仍可立即补入的下一请求。如果在整个统计与 drain 区间都没有观察到 prefix 尾端, 那么把任意长乃至无限的请求 suffix 接到这

32 个请求之后，不会改变该区间内的 admission、SSD 仲裁、NPU link、compute 或 TTFT 事件。因此通过 no_backlog_exhaustion 的正式行与无限 tail 在这个有限测量区间内因果等价。

这个结论不等于“已经模拟了无限时间”，也不自动证明队列分布严格 stationary；outstanding queue 的窗口趋势仍需单独报告。它只排除了“某个 NPU 因有限输入提前结束，SSU 带宽被错误让给其他 NPU”这一原始偏差。SSU16 admission 的 invalid 原因是 all_npus_sampled_for_slo=false，不是 prefix 耗尽，因而也没有用部分样本冒充正式结果。

7 128 NPU 最终配对结果

所有下列行均完整结束、通过 summary invariant，并保留成对 fingerprint。NPU-link 一列同时给出 summary 中的 NPU50 utilization 百分数和换算后的实际 GB/s，避免把百分数误读为带宽。

7.1 主结果

SSU	策略	Mean compute util	Equal-NPU 2× SLO	NPU-link max (%) NPU50 / GB/s)	Evals	Path writes
24	baseline	32.76%	25.00%	14.61% / 7.31	0	0
24	current Scheme B	32.38%	17.01%	15.37% / 7.68	43,759	410,103
24	V2 admis- sion25	32.51%	52.10%	21.12% / 10.56	249	754,209
24	Adaptive V2.1 25 ms	32.51%	52.10%	21.12% / 10.56	249	754,209
40	baseline	51.60%	50.00%	22.23% / 11.11	0	0
40	current Scheme B	51.57%	50.64%	24.04% / 12.02	47,492	4,041,483
40	V1 admis- sion25	51.57%	82.45%	24.83% / 12.41	166	844,631
40	V1 admis- sion50	51.70%	75.23%	25.52% / 12.76	85	430,120
40	V2 admis- sion25	51.66%	79.20%	24.36% / 12.18	162	821,977
40	Adaptive V2.1 25 ms	51.57%	82.45%	24.83% / 12.41	166	844,631

SSU	策略	Mean compute util	Equal-NPU 2× SLO	NPU-link max (%) NPU50 / GB/s)	Evals	Path writes
70	baseline	88.93%	77.33%	38.69% / 19.34	0	0
70	current Scheme B	89.06%	81.94%	38.97% / 19.48	53,902	6,019,200
70	V1 admis- sion25	90.85%	98.88%	38.82% / 19.41	131	611,236
70	V1 admis- sion50	90.78%	92.26%	38.98% / 19.49	65	362,115
70	V2 admis- sion25	90.89%	98.55%	38.91% / 19.45	131	585,135
70	Adaptive V2.1 25 ms	90.85%	98.88%	38.82% / 19.41	131	611,236

Adaptive 与所选 residual 的独立结果逐位一致：

SSU	Explicit/coflow evaluations	Last selected fraction	自动选择	对应结果
24	248 / 1	0.5859	V2 explicit spill	与 V2 相同：32.51% / 52.10%
40	0 / 166	0.9219	V1 coflow residual	与 V1 相同：51.57% / 82.45%
70	0 / 131	1.0000	V1 coflow residual	与 V1 相同：90.85% / 98.88%

Adaptive 和 V2 runner 均为 complete=true、selected_complete=true、source/config stable；V1 与 SSU24 supplemental 的已选正式行完成且 fingerprint 稳定。所有跨策略对比的五组输入指纹一致。

7.2 与同 SSU baseline 的配对差值

SSU	策略	Δ compute util	Δ equal-NPU SLO	判断
24	current Scheme B	-0.38 pp	-7.99 pp	旧策略反而变差
24	V2 / Adaptive V2.1	-0.25 pp	+27.10 pp	强 SLO 改善，mean util 保持
40	current Scheme B	-0.03 pp	+0.64 pp	旧目标函数没有形成大收益
40	V1 / Adaptive V2.1 25 ms	-0.03 pp	+32.45 pp	强 SLO 改善，mean util 保持
40	V1 50 ms	+0.11 pp	+25.23 pp	写入约减半，SLO 收益较低
40	V2 25 ms	+0.06 pp	+29.20 pp	强改善，但逊于 V1 SLO 3.25 pp
70	current Scheme B	+0.13 pp	+4.60 pp	改善有限
70	V1 / Adaptive V2.1 25 ms	+1.92 pp	+21.54 pp	SLO 与 mean util 同时改善
70	V1 50 ms	+1.85 pp	+14.93 pp	写入较少，SLO 收益较低
70	V2 25 ms	+1.96 pp	+21.22 pp	util 略高，SLO 略低于 V1

7.3 25 ms 与 50 ms 的性能/写入折中

SSU	Interval	Mean util	Equal-NPU SLO	Evals	Path writes	相对 25 ms
40	25 ms	51.57%	82.45%	166	844,631	基准
40	50 ms	51.70%	75.23%	85	430,120	util +0.13 pp, SLO -7.22 pp, writes -49.1%
70	25 ms	90.85%	98.88%	131	611,236	基准
70	50 ms	90.78%	92.26%	65	362,115	util -0.07 pp, SLO -6.62 pp, writes -40.8%

50 ms 将 evaluation 大致减半，mean utilization 几乎不变，但会漏掉更多短请求的 SLO 时机。若硬件 Path 写入成本是主要约束，50 ms 是有效降写入配置；若目标优先级是 TTFT SLO，25 ms 是正式结果支持的默认值。

SSU16 的独立 V1 admission25 诊断不能填入正式 util/SLO：唯一失败 invariant 是 `all_npus_sampled_for_slo=false`，其他 warmup、窗口、tag/drain、no_backlog_exhaustion、SSD/NPU 服务归属和容量检查都通过。报告没有把缺样本记成 0%，也没有用部分 NPU 均值替代正式结果。

7.4 128 与 32 NPU 不一致的已定位原因

- Ring ownership/placement 不等价：** 128×40 的 ring 最大 ownership 1.1696×，显著高于 32×10 的 1.0555×；容量比例相同不保持热点程度。
- NPU link 不是已完成行的主瓶颈：** 所有观测 max 不超过 38.98% of NPU50 = 19.49 GB/s，不能把 utilization 百分数直接写成 GB/s。
- 目标函数决定 binary SLO：** old Scheme B 在 SSU40 只有 +0.64 pp SLO，而 V1 admission 在几乎同一 mean util 下达到 +32.45 pp。
- Residual 模式依赖过载程度：** V2 在 SSU24 提供严重过载所需的 explicit spill，但在 SSU40 略逊于 V1；不能固定使用一种 tail 分配。
- 稳态证据仍需加强：** 多个已完成行存在 outstanding queue drift 或仅 compatible-not-proven；数值是有效窗口结果，但严格长期稳态结论要结合更长窗口/重复 block。
- 命令粒度不是当前差异解释：** 真实 SSD command 只有 2.10–4.20 μs，不足以单独制造几十毫秒差距。

下一轮 vnode/替代数据面实验仍必须保持 input/placement/source/config fingerprint 配对；不得在一个 runner 的长矩阵中修改源码后混用旧 checkpoint。

8 对“远优于 baseline”的物理解释

动态策略不能创造额外 SSD 或 NPU link 带宽。如果 baseline 已经接近容量给出的 mean utilization 上限，新策略不可能在 mean utilization 上再增加几十个百分点。合理的“远优”应定义为 Pareto 改善；Adaptive V2.1 在三个 128-NPU 容量点都达到这一标准：SSU24/40/70 分别为 -0.25/+27.10、-0.03/+32.45、+1.92/+21.54 pp (util/SLO)。

- 在接近 raw knee 的 SSU70，V1 已让 equal-NPU SLO 从 77.33% 提高到 98.88%，mean compute utilization 从 88.93% 提高到 90.85%；
- 在容量受限的 SSU24，Adaptive 已让 SLO 从 25.00% 提高到 52.10%，mean utilization 只变化 -0.25 pp；SSU40 则从 50.00% 提高到 82.45%，mean utilization 基本不变；
- 在所有配置中，吞吐与 SSD busy 不应出现无法解释的下降；
- 改善必须在同一 input/placement fingerprint、有效 steady window 和相同 request-ID 配对下成立。

也就是说，动态策略最大的价值是把有限服务机会放到能让请求跨过 barrier/SLO threshold 的位置，而不是把 40 GB/s 变成更大的数字。当前证据也否定了两个替代解释：这些行的 NPU link 没有接近 50 GB/s，真实单命令不可抢占时间只有微秒级；主要差异来自 placement hotspot、request-level threshold objective 和 residual service 的去向。

9 复现与文件索引

2×2 教学实验：

```
PYTHONDONTWRITEBYTECODE=1 python baseline_2npu_teaching_experiment.py
PYTHONDONTWRITEBYTECODE=1 pytest -q test_baseline_2npu_teaching_experiment.py
```

128 NPU 正式 runner 的冻结矩阵：

```
python steady_state_128npu_admission_experiment.py --help
python steady_state_128npu_admission_v2_experiment.py --help
python steady_state_128npu_adaptive_v2_1_experiment.py --help
```

本 PDF 的渲染命令：

```
pandoc BASELINE_QOS_STEADY_STATE_REPORT.md \
  --from markdown+raw_tex \
  --pdf-engine=xelatex \
  --output BASELINE_QOS_STEADY_STATE_REPORT.pdf
```

关键实现位置：

- policy_logic.py: baseline 固定 Path0;
- strategy_profiles.py: 静态 CIR、无限 PIR、group/path weight;
- sim.py: SSD40 单命令调度、虚拟完成标签、动态 CIR 生效语义;
- continuous_batch_sim.py: NPU50 FCFS 第二级服务与 steady-state 统计;
- baseline_2npu_teaching_experiment.py: 5 个 2×2 饱和输入与完整 invariant;
- adaptive_admission_scheme_b_v2_1.py: 按 selected fraction 切换 V1/V2 residual;
- steady_state_128npu_admission_experiment.py: 真实 data 的 128 NPU V1 冻结矩阵;
- steady_state_128npu_admission_v2_experiment.py: V2 explicit-tail 独立矩阵;
- steady_state_128npu_adaptive_v2_1_experiment.py: Adaptive V2.1 独立验证;
- results/steady_state_128npu_admission_v1/scale_analysis.md: 完成行、ring association、NPU-link 与 queue 稳态审计;
- results/ring_ownership_scale/analysis.md: 精确 ring ownership 积分。